

HealthBridge — How Android Code Interfaces with Firebase

This document explains how the HealthBridge Android application communicates with the Firebase Realtime Database, how Firebase nodes are created automatically, and how the Android code must match the Firebase structure exactly.

1. Firebase Realtime Database is Schemaless

Firebase Realtime Database does not require a predefined schema. The Android application can create nodes dynamically simply by writing data to a path.

Example:

```
groups/family_001/members/alain/latest/location/lat
```

If the path does not exist, Firebase automatically creates it.

2. The Android Code Creates Firebase Nodes

Example Kotlin write operation:

```
FirebaseDatabase.getInstance()
    .getReference("groups")
    .child("family_001")
    .child("members")
    .child("alain")
    .child("latest")
    .child("location")
    .child("lat")
    .setValue(latitude)
```

This automatically creates the entire node hierarchy if it does not already exist.

3. Read and Write Paths MUST Match

If the write structure is:

```
latest
  location
    lat
    lng
```

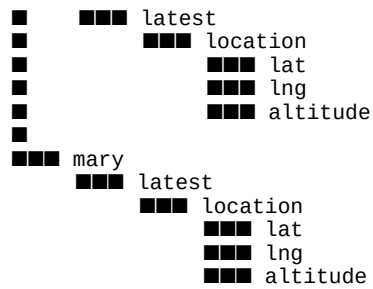
then the read code must use:

```
snapshot.child("latest")
    .child("location")
    .child("lat")
```

If the read path does not match exactly, Firebase returns null values. This may cause GPS coordinates to become 0.0 and the map to jump to Africa.

4. Current HealthBridge Firebase Structure

```
groups
  ■■■ family_001
    ■■■ members
      ■■■ alain
```



5. Publisher Flow

The publisher phone owns one member node.

Example:

- Alain phone writes ONLY to members/alain
- Mary phone writes ONLY to members/mary

Flow:

```
graph TD
    GC[GpsCollector] --> TE[TelemetryEngine]
    TE --> FM[FirebaseManager.updateLocation()]
    FM --> FRTD[Firestore Realtime Database]
```

Each phone publishes only its own GPS and watch data.

6. Viewer Flow

MainActivity creates listeners:

```
listenToMember("alain")
listenToMember("mary")
```

Each listener watches a Firestore node.

Whenever Firestore data changes, the callback triggers immediately and the map marker moves.

7. Why a Clean Schema is Important Before Adding the Watch

The watch will introduce additional data:

```
watch
{
  heartRate
  spo2
  battery
  steps
}
```

Without a clean structure:

- GPS and watch values become mixed
 - duplicate timestamps appear
 - debugging becomes difficult
 - scaling becomes dangerous
-

8. Recommended Future Structure

```
groups
  ■■■ family_001
    ■■■ members
      ■■■ alain
        ■■■ profile
          ■■■ name
          ■■■ latest
            ■■■ timestamp
            ■■■ location
              ■■■ lat
              ■■■ lng
              ■■■ altitude
            ■■■ watch
              ■■■ heartRate
              ■■■ spo2
              ■■■ steps
              ■■■ battery
          ■■■ device
            ■■■ phoneBattery
            ■■■ lastSeen
```

9. Important Synchronization Rule

Only ONE phone should publish data for one member.

Example:

- Alain phone publishes ONLY Alain
- Mary phone publishes ONLY Mary

No phone should write another member's node.

This prevents synchronization conflicts and inconsistent Firebase states.

10. Firebase Updates are Asynchronous

Firebase writes and listener updates are asynchronous.

Updates may arrive milliseconds later and listeners may trigger in different orders.

Therefore:

- one publisher per member
- one source of truth per member

is essential for HealthBridge stability.
